

Topology Statistics Plotting Specification

Normative TS5 API for Catalog-Derived Statistical Figures in mdstats

mdstats

2026-07-14 (implemented TS5 revision)

Contents

Purpose and status	1
Motivation	1
Public types	2
Public functions	2
Atomic pair-count distribution	2
Atomic pair-count series	2
Catalog occupancy	2
Catalog assignment series	3
Transition raster	3
Transition matrix	3
Residence-length distribution	4
Contact-occupancy distribution	4
Framework graph-descriptor series	4
Cross-layer plots	4
Input constraints	5
Output contract	5
Algorithmic behavior	5
Edge cases and warnings	5
Validation requirements	6

Purpose and status

This document specifies the implemented module

`mdstats/plotting/topology_statistics.py`

introduced in `mdstats 0.17.0a5` as `topology-statistics` Stage TS5.

The module converts completed TS0–TS4 result objects into Matplotlib figures. It never reads atomic coordinates, rebuilds connectivity, projects framework edges, recomputes graph descriptors, or changes catalog identity.

The governing boundary is

statistics results define values; plotting only presents them .

Motivation

Topology statistics contain exact discrete distributions, state assignments, transition events, graph descriptors, and cross-layer contingency matrices. These objects are machine-readable but are not by themselves convenient for scientific inspection.

A standard plotting layer provides:

- reproducible figures from one authoritative result;
- consistent axis semantics across trajectories and ensembles;
- exact integer probability-mass plots instead of arbitrary binning;
- visible separation between atomic contacts and projected framework topology;
- direct access to Matplotlib Figure and Axes objects for downstream styling.

Public types

```
ProbabilityUnit = Literal["fraction", "percent"]
TopologyBranch = Literal["atomic", "framework"]
```

Accepted result types are:

```
AtomicConnectivityStatistics
FrameworkTopologyStatistics
TopologyStatistics
```

A TopologyStatistics object exposes both branches. Functions that require one branch accept branch="atomic" or branch="framework".

Public functions

Atomic pair-count distribution

```
def plot_pair_count_distribution(
    result: AtomicConnectivityStatistics | TopologyStatistics,
    left: int | str,
    right: int | str,
    *,
    ax: Axes | None = None,
    probability_unit: Literal["fraction", "percent"] = "fraction",
    title: str | None = None,
    annotate_summary: bool = True,
) -> tuple[Figure, Axes]:
    ...
```

The function plots the exact PMF

$$p_{AB}(n) = \frac{1}{F} \sum_{f=1}^F \mathbf{1}[N_{AB}(f) = n].$$

Each supported integer count is one bar. No continuous histogram or smoothing is applied.

Atomic pair-count series

```
def plot_pair_count_timeseries(
    result: AtomicConnectivityStatistics | TopologyStatistics,
    left: int | str,
    right: int | str,
    *,
    ax: Axes | None = None,
    title: str | None = None,
) -> tuple[Figure, Axes]:
    ...
```

The x-axis is selected from the authoritative FrameAxis:

1. physical time when available;
2. simulation step otherwise;
3. frame index for a trajectory;
4. sample index for an ensemble.

An ensemble plot is a sample-index series, not a temporal trajectory.

Catalog occupancy

```
def plot_catalog_state_occupancy(
    result: AtomicConnectivityStatistics
        | FrameworkTopologyStatistics
        | TopologyStatistics,
    *,
```

```

branch: Literal["atomic", "framework"] = "atomic",
ax: Axes | None = None,
probability_unit: Literal["fraction", "percent"] = "fraction",
title: str | None = None,
) -> tuple[Figure, Axes]:
    ...

```

For catalog state or topology class s ,

$$p_s = \frac{n_s}{F}.$$

The function plots one bar per dense state or class ID.

Catalog assignment series

```

def plot_catalog_state_timeline(
    result: AtomicConnectivityStatistics
        | FrameworkTopologyStatistics
        | TopologyStatistics,
    *,
    branch: Literal["atomic", "framework"] = "atomic",
    ax: Axes | None = None,
    title: str | None = None,
) -> tuple[Figure, Axes]:
    ...

```

The function uses a step plot of exact frame-to-state assignments. For ensembles, the title and x-axis identify sample order and do not imply dynamics.

Transition raster

```

def plot_transition_raster(
    result: AtomicConnectivityStatistics
        | FrameworkTopologyStatistics
        | TopologyStatistics,
    *,
    branch: Literal["atomic", "framework"] = "atomic",
    ax: Axes | None = None,
    title: str | None = None,
) -> tuple[Figure, Axes]:
    ...

```

Every changed-state boundary is shown at its exact right-hand frame coordinate. The function requires TS3 StateTransitionStatistics. It rejects ensemble or option-disabled results whose temporal branch is absent.

Transition matrix

```

def plot_transition_matrix(
    result: AtomicConnectivityStatistics
        | FrameworkTopologyStatistics
        | TopologyStatistics,
    *,
    branch: Literal["atomic", "framework"] = "atomic",
    changed_only: bool = True,
    ax: Axes | None = None,
    title: str | None = None,
) -> tuple[Figure, Axes]:
    ...

```

The default matrix is

$$M_{ij} = \#\{f : s_f = i, s_{f+1} = j, i \neq j\}.$$

When `changed_only=False`, diagonal adjacent-state counts are retained.

Residence-length distribution

```
def plot_dwelling_distribution(  
    result: AtomicConnectivityStatistics  
        | FrameworkTopologyStatistics  
        | TopologyStatistics,  
    *,  
    branch: Literal["atomic", "framework"] = "atomic",  
    ax: Axes | None = None,  
    probability_unit: Literal["fraction", "percent"] = "fraction",  
    title: str | None = None,  
) -> tuple[Figure, Axes]:  
    ...
```

The function plots the exact PMF of maximal residence interval lengths in frames. It does not estimate a kinetic lifetime distribution.

Contact-occupancy distribution

```
def plot_contact_occupancy_distribution(  
    result: AtomicConnectivityStatistics | TopologyStatistics,  
    left: int | str,  
    right: int | str,  
    *,  
    ax: Axes | None = None,  
    bins: int = 20,  
    title: str | None = None,  
) -> tuple[Figure, Axes]:  
    ...
```

The underlying values are gauge-invariant atom-pair occupancies

$$p_c = \frac{1}{F} \sum_f \mathbf{1}[c \in E_f].$$

Unlike integer count PMFs, occupancy is continuous on $[0, 1]$ and is displayed as a histogram.

Framework graph-descriptor series

```
def plot_graph_descriptor_timeseries(  
    result: FrameworkTopologyStatistics | TopologyStatistics,  
    descriptor: str,  
    *,  
    ax: Axes | None = None,  
    title: str | None = None,  
) -> tuple[Figure, Axes]:  
    ...
```

Valid descriptor names are those stored by TS2, including vertex count, edge count, component count, isolated-vertex count, self-image-edge count, parallel edge statistics, and cycle-space rank

$$\beta_1 = E - V + C.$$

The quantity β_1 must not be labeled as a primitive-ring count.

Cross-layer plots

```
def plot_cross_layer_boundary_counts(  
    result: TopologyStatistics,  
    *,  
    ax: Axes | None = None,  
    title: str | None = None,  
) -> tuple[Figure, Axes]:  
    ...
```

The four categories are stable, atomic-only, framework-only, and coupled.

```
def plot_cross_layer_contingency(
    result: TopologyStatistics,
    *,
    ax: Axes | None = None,
    probability: bool = False,
    title: str | None = None,
) -> tuple[Figure, Axes]:
    ...
```

The displayed matrix is either the exact frame-count matrix C_{ak} or its collection-wide probability matrix.

Input constraints

- Inputs must be completed, internally validated TS0–TS4 result objects.
- Species identifiers must resolve to a pair present in the atomic result.
- branch must be atomic or framework.
- Temporal plots require a trajectory result with detailed temporal statistics.
- Contact-occupancy plots require occupancy calculation to have been enabled.
- Existing axes must be Matplotlib Axes objects.
- Probability units are fractions or percentages only.

Output contract

Every function returns

(Figure, Axes)

The caller owns saving, further styling, and closing the figure. Plotting functions must not call `show()` or write files implicitly.

Algorithmic behavior

Plotting cost is linear in the number of presented values. For F frames, S states, and B nonzero matrix entries, representative scaling is:

Plot	Time	Memory
Pair series	$O(F)$	$O(F)$ renderer input
Exact PMF	$O(K)$	$O(K)$
State occupancy	$O(S)$	$O(S)$
Transition raster	$O(T)$	$O(T)$
Transition matrix	$O(S^2)$	$O(S^2)$
Contingency matrix	$O(N_A N_F)$	$O(N_A N_F)$

No plot should expand graph edges or rerun catalog calculations.

Edge cases and warnings

- A constant series is valid and should appear as a flat line or one-bin PMF.
- A zero-transition trajectory produces an empty raster without an exception.
- Large state matrices may be visually dense even when scientifically valid.
- State IDs are catalog-local labels, not physically ordered coordinates.
- Sample-index ensemble plots must not be interpreted as transition sequences.
- Na–O contacts may be coordination contacts rather than chemical bonds.
- A framework edge count can remain constant while decorated edge identity changes; transition plots, not only scalar counts, must be inspected in reactive systems.

Validation requirements

Tests must verify:

- plotted values equal the source result arrays exactly;
- exact integer PMF bar heights;
- axis-label selection from `FrameAxis`;
- temporal plot rejection when temporal statistics are absent;
- support for caller-provided axes;
- PNG, SVG, and PDF figure export by Matplotlib;
- framework descriptor and cross-layer matrix dimensions;
- no mutation of source statistics objects.